



تمرین ۱ بخش ۴ تئوری

استاد:

دکتر امینی، دکتر خرازی

نویسنده:

پارمیس حمصیان اتفاق

(الف) برای اینکه یک نفر مجاز به خواندن یک فایل باید در شرایط که BLP هست باید هم سطح یا بالاتر از فایل باشد و برای نوشتن باید هم سطح و یا پایین تر از سطح فایل باشد علاوه بر آن باید این دسترسی را داشته باشد.

1. مجاز - با توجه به هم سطح بودن فایل و خواننده و همچنین وجود دسترسی خواندن برای آلیس خواندن فایل امکان پذیر است.
2. غیر مجاز - پایین تر بودن میزان دسترسی آلیس نسبت به فایل (no read up)
3. غیر مجاز - بدلیل وجود شرط no write down و اینکه دسترسی باب top secret هست و فایل secret این امکان وجود ندارد.

(ب) برای اینکه یک نفر مجاز به خواندن یک فایل باید در شرایط که Biba هست باید هم سطح یا پایین تر از فایل باشد و برای نوشتن باید هم سطح و یا بالاتر از سطح فایل باشد علاوه بر آن باید این دسترسی را داشته باشد.

1. غیر مجاز - همانطور که در بالا توضیح داده شد سطح فرد باید پایین تر از فایل برای خواند باشد در صورتی که دانا در high و بالاتر از سطح فایل قرار دارد.
2. مجاز - با توجه به پایین تر بودن سطح دسترسی کاملاً امکان پذیر هست.
3. مجاز - با توجه به اینکه سطح دسترسی کارول بالاتر است این موضوع مشکلی ندارد.

(ج) برای اینکه در این شرایط مجاز بودن یک عملیات تنها در صورتی که هر دو تا شرط بالا اعمال بشه امکان پذیر هست.

1. غیر مجاز - شرایط BLP با توجه به اینکه برای خواندن باید سطح مساوی و بالاتر از فایل باشه برای آلیس امکان پذیر نیست.
2. مجاز - هر دو شرط برقرار هستند.
3. غیر مجاز - با توجه به شرط no write down در BLP و بالاتر بودن سطح دانا از فایل این شرط برقرار نمی شود.

(د) برای این بخش علاوه بر شرایطی که در قسمت الف و ب ذکر شد باید role هر فرد رو بررسی کنیم که یکسان باشد.

1. مجاز - هم Analyst هست و هم در BLP در یک سطح قرار دارند و در Biba در سطح پایین تر.
2. غیر مجاز - role کارول جزو افرادی که امکان خواندن دارند حساب نمی شود علاوه بر آن شرایط Biba را نیز امکان ندارد.

3. مجاز - تمامی موارد در یک سطح هستند و همه موارد مجاز هست.

ه) با توجه به شرایطی که در بخش های قبل گفته شده هر بخش را بررسی می کنیم:

1. امکان پذیر نیست - با توجه به اینکه در BLP باید no read up باشد و آلیس و فایل F_1 هر دو در یک سطح هستند این بخش امکان پذیر هست. بخش بعد نیز چون هم سطح هستند باز امکان پذیر هست. در بخش آخر کارول امکان خواند فایل را ندارد چون در یک سطح پایین تر قرار دارد.
 2. در صورتی که تخت قوانین BLP بررسی شود فایلی که آلیس می نویسد اگر confidential باشد کارول امکان خواندنش را دارد اما کارول امکان نوشتن در سطح پایین تر را ندارد پس این امکان برقرار نمی شود
 3. آلیس می تواند فایل را بخواند چون در یک سطح قرار دارند، امکان نوشتن نیز دارد چون باز در یک سطح هستند و چون در Biba، no read down فعال هست پس کارول امکان خواندن فایل را ندارد و برای بخش بعد تنها در صورتی کارول می تواند فایل را بخواند که در سطح critical باشد ولی آلیس در Biba به دلیل وجود no write up امکان نوشتن فایل با برچسب critical را ندارد.
- با تحلیل این دو بخش نتیجه می گیریم در صورتی که در مرحله اول شرایط امکان پذیر نباشد با تغییر برچسب نیز امکان پذیر نخواهد بود.

الف) در این سناریو، فرستنده ابتدا D100 تا D400 را می‌فرستد (با توجه به پنجره ۳۰۰ بایتی). چون D200 گم می‌شود، فرستنده در نهایت تایم‌اوت داده و دوباره D200 را می‌فرستد. بعد از تایید آن، پنجره جلو رفته و بسته‌های بعدی ارسال می‌شوند. همین اتفاق برای D700 هم تکرار می‌شود.

#	Packet Data Offset	Sent On Timeout	Dropped	Cumulative Ack
1	100			A200
2	200		X	
3	300			A200
4	400			A200
5	200	X		A500
6	500			A600
7	600			A700
8	700		X	A700
9	800			A700
10	900			A700
11	700	X		A1000
12	1000FIN			A1100FIN

ب) زمان کل شامل دو وقفه طولانی RTO (هر کدام ۳ ثانیه) برای بسته‌های گم شده و مجموعه‌ای از RTT ها برای رفت و برگشت بسته‌های سالم است. در مجموع ۶.۰۴ ثانیه زمان می‌برد تا تمام ۱۰۰۰ بایت تایید شوند.

(الف) خیر، اگر ما فقط بسته‌هایی که شماره آن‌ها توان ۲ است (مثل ۱، ۲، ۴، ۸، ...) را تایید (ACK) کنیم، فرستنده هیچ راهی ندارد که بفهمد بسته‌های بین این اعداد (مثلاً بسته‌های ۳، ۵، ۶ و ۷) سالم به مقصد رسیده‌اند یا در شبکه گم شده‌اند. در لایه انتقال، تاییدیه باید به گونه‌ای باشد که وضعیت تک‌تک بایت‌ها یا بسته‌ها مشخص شود.

(ب) برای اینکه این پروتکل Reliable شود، باید از مکانیزم Cumulative ACK استفاده کنیم. یعنی وقتی گیرنده ACK شماره ۸ را می‌فرستد، این پیام باید به این معنا باشد که: «من تمام بسته‌های ۱ تا ۸ را به طور کامل و صحیح دریافت کرده‌ام». در این صورت، حتی اگر فقط توان‌های ۲ ارسال شوند، باز هم وضعیت بسته‌های میانی برای فرستنده روشن می‌شود.

(ج) خیر، این پروتکل نه تنها باعث صرفه‌جویی نمی‌شود، بلکه به شدت پهنای باند را هدر می‌دهد. دلیلش این است که وقتی پنجره ارسال Window Size به صورت نمایی بزرگ می‌شود (در توان‌های ۲)، حجم عظیمی از بسته‌ها به صورت همزمان وارد شبکه می‌شوند. این ترافیک شدید باعث پر شدن بافر روترها و Congestion می‌شود. در این حالت اگر حتی یک بسته از این حجم عظیم گم شود، چون تاییدیه فقط در توان‌های ۲ صادر می‌شود، فرستنده مجبور است تمام آن چند صد هزار بسته را دوباره ارسال کند که این یعنی یک فاجعه در مصرف پهنای باند.

(د) این طرح در حالت عادی Reliable نیست. یک باگ منطقی بزرگ دارد: اگر تعداد کل بسته‌های ارسالی فرد باشد، بسته آخر هیچ‌گاه تایید (ACK) نمی‌شود. چون گیرنده طبق پروتکل مریم، فقط منتظر بسته‌های زوج می‌ماند تا تاییدیه بفرستد و چون بعد از بسته آخر (که فرد است) دیگر بسته زوجی نمی‌آید، سیستم در یک بن بست گیر می‌کند.

ه) برای حل مشکل بالا، فرستنده باید شمارنده بسته‌ها را چک کند. اگر متوجه شد که تعداد کل بسته‌ها فرد است، بعد از فرستادن آخرین بسته اصلی، یک بسته صوری یا پوچ (Dummy Packet) ارسال کند. این بسته دیتای خاصی ندارد و فقط برای این فرستاده می‌شود که شماره ترتیب را زوج کند تا گیرنده تحریک شده و ACK نهایی را صادر کند.

و) بله، در این طرح به طور میانگین تعداد تاییدیه‌های ارسالی کمتر از حالت استاندارد (یک ACK برای هر بسته) است.

اثبات: فرض کنید p احتمال گم شدن یک بسته باشد. در حالت معمولی، برای هر بسته یک تلاش بر اساس توزیع هندسی داریم. در طرح مریم، موفقیت زمانی حاصل می‌شود که هر دو بسته یک جفت سالم برسند، یعنی با احتمال $(1 - p)^2$. البته احتمال نیاز به بازفرستادن در این جفت‌ها بیشتر است، اما چون به ازای هر دو بسته فقط یک ACK تولید می‌شود، در نهایت مجموع تعداد سیگنال‌های ACK در کل شبکه نسبت به حالت عادی کاهش می‌یابد.

الف) با توجه به صورت سوال خواهیم داشت:

1. ابتدا کاربر C یک اتصال معمولی به سرور A برقرار می‌کند تا بفهمد سیستم شماره‌گذاری A چگونه کار می‌کند. فرض کنید شماره دنباله‌ای که دریافت می‌کند SN_1 باشد. چون این شماره‌ها اغلب بر اساس ساعت سیستم تولید می‌شوند، C می‌تواند نرخ افزایش آن را بفهمد.
2. حالا C یک بسته SYN به سرور A می‌فرستد، اما در قسمت IP مبدا، آدرس کاربر B را جعل می‌کند.
3. سرور A به خیال اینکه B درخواست اتصال داده، یک بسته $SYN+ACK$ حاوی شماره دنباله جدید (SN_2) برای B می‌فرستد. طبق فرض، B این بسته را نادیده می‌گیرد یا اصلاً آن را دریافت نمی‌کند.
4. نفوذگر C که بسته قبلی را ندیده، باید SN_2 را حدس بزند. او با استفاده از SN_1 و تخمین زمانی که گذشته (RTT)، یک عدد حدسی برای ACK تولید کرده و همراه با دیتای مخرب به A می‌فرستد. اگر حدس او درست باشد، A تصور می‌کند اتصال با B برقرار شده و دیتای مخرب را پردازش می‌کند.
5. پس از ارسال دیتای مخرب، C دیگر نیازی به ادامه ارتباط ندارد و اتصال پس از مدتی نیمه‌باز ماندن، قطع می‌شود؛ اما تخریب مورد نظر قبلاً انجام شده است.

ب) اگر فرض کنیم زمان رفت و برگشت (RTT) حدود ۴۰ میلی‌ثانیه باشد، باید تعداد شماره‌های ممکن در این بازه را محاسبه کنیم.

با توجه به اینکه شماره دنباله در هر ۴ میکروثانیه یک واحد افزایش می‌یابد، در یک بازه زمانی ۴۰ میلی‌ثانیه‌ای (که زمان تقریبی برای یک RTT است)، حدود ۱۰۰۰۰ شماره دنباله مختلف می‌تواند تولید شود. بنابراین، نفوذگر به طور میانگین باید ۱۰۰۰۰ بار تلاش کند تا بتواند شماره دنباله دقیق را حدس زده و حمله را با موفقیت کامل کند.

سوال ۵

(الف)

- نمودار ۱: (TCP Tahoe) در این نمودار می‌بینیم که هر بار اتلاف رخ داده (چه با دریافت ۳ تا ACK تکراری و چه تایم‌اوت)، $cwnd$ بلافاصله به مقدار ۱ ریست شده و فاز Slow Start دوباره از اول شروع شده که ویژگی اصلی تاهو است.
- نمودار ۲: (TCP Reno) در اینجا سیستم هوشمندتر عمل کرده. وقتی ۳ تا ACK تکراری دریافت شده (Triple Duplicate ACKs)، پنجره به جای ریست شدن، به نصف مقدار قبلی کاهش پیدا کرده (Fast Recovery) و فقط وقتی تایم‌اوت واقعی رخ داده، به ۱ برگشته. این رفتار نسخه رنو است.

(ب)

نمودار مربوطه	نوع رویداد	تحلیل
۱ (Tahoe)	Loss (3-Dup ACK / TO	در تاهو فرقی بین نوع اتلاف نیست؛ $ssthresh$ نصف شده و $cwnd$ به ۱ سقوط می‌کند تا فاز Slow Start مجدداً آغاز شود.
۱ (Tahoe)	Threshold Reach	مقدار $cwnd$ به $ssthresh$ رسیده است. رفتار از رشد نمایی به رشد خطی (Congestion Avoidance) تغییر می‌کند.
۱ (Tahoe)	Loss (Severe	تکرار سناریوی نقطه ۱؛ به دلیل اتلاف، پنجره مجدداً به مقدار ۱ ریست شده است.
۲ (Reno)	Threshold Reach	مشابه تاهو، با رسیدن به آستانه، رشد پنجره برای احتیاط بیشتر به صورت خطی (Additive Increase) ادامه می‌یابد.
۲ (Reno)	Triple Duplicate ACK	سناریو: دریافت ۳ عدد ACK تکراری. واکنش: اجرای Fast Recovery؛ $ssthresh$ نصف شده و $cwnd$ برخلاف تاهو، به جای ۱ برابر با $ssthresh$ قرار می‌گیرد.
۲ (Reno)	Timeout	سناریو: انقضای تایمر (عدم دریافت هیچ ACK). واکنش: چون شبکه به شدت شلوغ فرض می‌شود، Reno پنجره را به ۱ ریست می‌کند.
۳ (Cubic)	Multi-step Loss	سناریو: اتلاف در پهنای باند بالا. واکنش: کاهش پنجره طبق ضابطه کنترل Cubic (معمولاً با ضریب ۰.۷) و شروع فاز رشد مقعر برای بازگشت سریع به W_{max} .

سناریو: عبور از سطح قبلی W_{max} . واکنش: تغییر فاز از حالت مقعر به محدب (Convex) برای جستجوی پهنای باند جدید در شبکه.	Probing New BW	۳ (Cubic)
--	-------------------	-----------

(ج)

الگوریتم (نمودار)	مزیت	عیب
TCP Tahoe (نمودار ۱)	سادگی و پایداری بالا: به دلیل رفتار محافظه کارانه و ریست کردن پنجره به ۱، احتمال ایجاد congestion مجدد در شبکه را به حداقل می‌رساند.	بهره‌وری پایین (Low Throughput): با هر بار گم شدن بسته، زمان زیادی را در فاز Slow Start هدر می‌دهد و پهنای باند را به خوبی پر نمی‌کند.
TCP Reno (نمودار ۲)	ریکاوری سریع (Fast Recovery): با نصف کردن پنجره در صورت دریافت ۳ عدد ACK تکراری، از شروع مجدد از پله اول جلوگیری کرده و گذردهی را بالا نگه می‌دارد.	ضعف در برابر خطاهای تجمعی: اگر چندین بسته از یک پنجره همزمان گم شوند، Reno به اشتباه دچار چندین کاهش متوالی شده یا در نهایت به Timeout منجر می‌شود.
TCP CUBIC (نمودار ۳)	مقیاس‌پذیری در پهنای باند بالا: به دلیل رشد تابع درجه ۳، خیلی سریع فضاهای خالی لینک‌های پرسرعت (مثل فیبر نوری) را پر می‌کند.	عدم انصاف (Fairness Issues): در رقابت با نسخه‌های قدیمی مثل Reno، ممکن است سهم بیشتری از پهنای باند را تصاحب کرده و باعث گرسنگی (Starvation) آن‌ها شود.

(د) در بین این سه مورد، TCP CUBIC تنها جمی‌تر عمل می‌کند در حالی که رنو و تاهو بعد از هر کاهش، با شیب ثابت و کند به مسیر برمی‌گردند، CUBIC به گونه‌ای طراحی شده که زمان بسیار کمتری را در پنجره‌های کوچک سپری کند. این پروتکل با استفاده از تابع پتانسیل، خیلی سریع فضای خالی لینک را پر می‌کند و میانگین گذردهی (Throughput) بالاتری را نسبت به بقیه ثبت می‌کند. البته تاثیر منفی در باقی قسمت‌ها ممکن است داشته باشد مثل انصاف و تاخیر.

سوال ۵

(الف) با بررسی ۸ بایت اول هر سگمنت (که مربوط به هدر UDP است)، اطلاعات زیر استخراج می‌شود:

• سگمنت A:

- بایت‌های اول (Source Port): 0xc001 که در مبنای ۱۰ برابر با ۴۹۱۵۳ است.
- بایت‌های دوم (Dest Port): 0x0035 که در مبنای ۱۰ برابر با ۵۳ است.
- نتیجه: این سگمنت یک درخواست (Query) از سمت کلاینت به سرور DNS است.

• سگمنت B:

- بایت‌های اول (Source Port): 0x0035 یعنی پورت ۵۳.
- بایت‌های دوم (Dest Port): 0xc001 یعنی پورت ۴۹۱۵۳.
- نتیجه: این سگمنت پاسخ (Response) سرور DNS به کلاینت است.

(ب) طول سگمنت در بایت‌های ۵ و ۶ هدر مشخص می‌شود:

- در سگمنت A مقدار طول 0x0018 است که یعنی ۲۴ بایت (۸ بایت هدر + ۱۶ بایت دیتا).
- در سگمنت B مقدار طول 0x0038 است که یعنی ۵۶ بایت (۸ بایت هدر + ۴۸ بایت دیتا).

(ج) طول سگمنت د

1. وضعیت سگمنت A: مقدار چک‌سام در این سگمنت 0x0000 است. در استاندارد UDP برای IPv4، اگر فرستنده نخواهد چک‌سام را محاسبه کند، تمام بیت‌های این فیلد را صفر می‌گذارد. بنابراین در سگمنت A، عملیات چک‌سام توسط فرستنده انجام نشده است.

2. وضعیت سگمنت B : در این سگمنت مقدار چک سام برابر با 0x9F10 است. برای اینکه بفهمیم این مقدار درست است یا خیر، باید طبق الگوریتم Checksum عمل کنیم.

$$0A00 + 0001 + C0A8 + D57D + 0011 + 0040$$

اگر تمام این اعداد ۱۶ بیتی را در مبنای ۱۶ با هم جمع کنیم carry حاصل را دوباره به انتهای عدد اضافه کنیم، حاصل نهایی باید برابر با FFFF می شود. از آنجا که حاصل جمع نهایی برابر با FFFF (یعنی تماماً ۱ در مبنای ۲) شده است، نتیجه می گیریم که مقدار چک سام کاملاً درست محاسبه شده و داده های سگمنت B بدون هیچ خطایی به مقصد رسیده اند.